

Model and Variable Reduction Comparison

August 6, 2024

1 PCA vs EFA Comparisons

```
[ ]: import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.feature_selection import RFE
from sklearn.metrics import accuracy_score, classification_report, \
    ↪confusion_matrix
from factor_analyzer import FactorAnalyzer

# Load the dataset
file_path = '/Users/hunterberberich/Downloads/cleaned_data_survive.csv'
data = pd.read_csv(file_path)

# Preparing the data
X = data.drop(columns=['date', 'survive'])
y = data['survive']

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split the data for evaluation
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, \
    ↪random_state=42)

[ ]: # PCA
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

rfc_pca = RandomForestClassifier(n_estimators=100, random_state=42)
```

```

rfc_pca.fit(X_train_pca, y_train)
y_pred_pca = rfc_pca.predict(X_test_pca)

print("PCA Model")
print("Accuracy:", accuracy_score(y_test, y_pred_pca))
print("Classification Report:\n", classification_report(y_test, y_pred_pca))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_pca))

```

PCA Model

Accuracy: 0.7868068833652008

Classification Report:

	precision	recall	f1-score	support
0.0	0.13	0.05	0.07	172
1.0	0.83	0.93	0.88	874
accuracy			0.79	1046
macro avg	0.48	0.49	0.48	1046
weighted avg	0.72	0.79	0.75	1046

Confusion Matrix:

```

[[ 9 163]
 [ 60 814]]

```

```

[ ]: # RFE
rfc = RandomForestClassifier(n_estimators=100, random_state=42)
rfe = RFE(estimator=rfc, n_features_to_select=10, step=1)
rfe.fit(X_train, y_train)
X_train_rfe = rfe.transform(X_train)
X_test_rfe = rfe.transform(X_test)

rfc_rfe = RandomForestClassifier(n_estimators=100, random_state=42)
rfc_rfe.fit(X_train_rfe, y_train)
y_pred_rfe = rfc_rfe.predict(X_test_rfe)

print("\nRFE Model")
print("Accuracy:", accuracy_score(y_test, y_pred_rfe))
print("Classification Report:\n", classification_report(y_test, y_pred_rfe))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rfe))

```

RFE Model

Accuracy: 0.7887189292543021

Classification Report:

	precision	recall	f1-score	support
0.0	0.14	0.06	0.08	172
1.0	0.83	0.93	0.88	874

accuracy			0.79	1046
macro avg	0.49	0.50	0.48	1046
weighted avg	0.72	0.79	0.75	1046

Confusion Matrix:

```
[[ 10 162]
 [ 59 815]]
```

```
[ ]: # EFA
fa = FactorAnalyzer(n_factors=10, rotation='varimax')
fa.fit(X_train)
X_train_efa = fa.transform(X_train)
X_test_efa = fa.transform(X_test)

rfc_efa = RandomForestClassifier(n_estimators=100, random_state=42)
rfc_efa.fit(X_train_efa, y_train)
y_pred_efa = rfc_efa.predict(X_test_efa)

print("\nEFA Model")
print("Accuracy:", accuracy_score(y_test, y_pred_efa))
print("Classification Report:\n", classification_report(y_test, y_pred_efa))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_efa))
```

EFA Model

Accuracy: 0.7896749521988528

Classification Report:

	precision	recall	f1-score	support
0.0	0.15	0.06	0.08	172
1.0	0.83	0.93	0.88	874

accuracy			0.79	1046
macro avg	0.49	0.50	0.48	1046
weighted avg	0.72	0.79	0.75	1046

Confusion Matrix:

```
[[ 10 162]
 [ 58 816]]
```

```
[ ]: # Factor Loadings from EFA
loadings = fa.loadings_
features = X.columns
loadings_df = pd.DataFrame(loadings, index=features, columns=[f'Factor {i+1}'
↪for i in range(loadings.shape[1])])

print("Factor Loadings from EFA:")
```

```

print(loadings_df)

most_influential_variables = loadings_df.apply(lambda row: row.abs().
    ↪nlargest(1).index, axis=1)
print("\nMost Influential Variables in Each Factor:")
print(most_influential_variables)

# Additional Models using EFA features
models_performance = []

lr = LogisticRegression(random_state=42)
lr.fit(X_train_efa, y_train)
y_pred_lr = lr.predict(X_test_efa)
models_performance.append(('Logistic Regression', accuracy_score(y_test, ↪
    ↪y_pred_lr), classification_report(y_test, y_pred_lr), ↪
    ↪confusion_matrix(y_test, y_pred_lr)))

rfc = RandomForestClassifier(n_estimators=100, random_state=42)
rfc.fit(X_train_efa, y_train)
y_pred_rfc = rfc.predict(X_test_efa)
models_performance.append(('Random Forest', accuracy_score(y_test, y_pred_rfc), ↪
    ↪classification_report(y_test, y_pred_rfc), confusion_matrix(y_test, ↪
    ↪y_pred_rfc)))

gbc = GradientBoostingClassifier(n_estimators=100, random_state=42)
gbc.fit(X_train_efa, y_train)
y_pred_gbc = gbc.predict(X_test_efa)
models_performance.append(('Gradient Boosting', accuracy_score(y_test, ↪
    ↪y_pred_gbc), classification_report(y_test, y_pred_gbc), ↪
    ↪confusion_matrix(y_test, y_pred_gbc)))

svm = SVC(random_state=42)
svm.fit(X_train_efa, y_train)
y_pred_svm = svm.predict(X_test_efa)
models_performance.append(('SVM', accuracy_score(y_test, y_pred_svm), ↪
    ↪classification_report(y_test, y_pred_svm), confusion_matrix(y_test, ↪
    ↪y_pred_svm)))

for model_name, accuracy, report, cm in models_performance:
    print(f"{model_name} Model")
    print("Accuracy:", accuracy)
    print("Classification Report:\n", report)
    print("Confusion Matrix:\n", cm)
    print("\n")

```

Factor Loadings from EFA:

Factor 1 Factor 2 Factor 3 Factor 4 \

tempmax	0.957705	-0.009826	0.018948	-0.202304
tempmin	0.959915	0.041289	0.067613	0.056086
temp	0.976917	0.008846	0.034195	-0.093894
feelslikemax	0.953900	-0.003071	0.025297	-0.185251
feelslikemin	0.959501	0.030740	0.064007	0.041668
feelslike	0.973346	0.010090	0.036975	-0.093879
dew	0.947705	0.128523	0.092608	0.217041
humidity	0.121432	0.403565	0.182325	0.736000
precip	0.103505	0.663745	0.130795	0.156125
precipcover	-0.017725	0.936445	0.221567	0.161547
snow	-0.158423	0.032949	0.045024	0.030994
snowdepth	-0.372413	-0.035447	-0.017106	0.106082
cloudcover	-0.279857	0.367064	0.293365	0.603841
uvindex	0.523918	-0.278194	-0.064896	-0.542021
severerisk	0.163762	-0.016863	0.123049	-0.038223
num_date	0.222977	-0.020721	-0.041191	0.048688
preciptype_freezingrain	-0.027060	-0.014204	0.011068	-0.040755
preciptype_freezingrain,snow	-0.050070	-0.041820	-0.017064	0.112426
preciptype_freezingrain,snow,ice	-0.038346	0.057958	-0.019849	0.016594
preciptype_none	0.006298	-0.336231	-0.823067	-0.262632
preciptype_rain	0.298778	0.290935	0.845984	0.083096
preciptype_rain,freezingrain	-0.007784	0.009922	-0.008296	0.010844
preciptype_rain,freezingrain,snow	-0.018340	-0.010196	-0.005271	0.006140
preciptype_rain,snow	-0.334521	0.063671	-0.016746	0.151155
preciptype_rain,snow,ice	-0.054004	0.002579	0.004724	0.004994
preciptype_snow	-0.287482	-0.021422	0.018183	-0.001055

	Factor 5	Factor 6	Factor 7	Factor 8	\
tempmax	-0.074919	-0.088413	-0.117957	0.038336	
tempmin	-0.057100	-0.063607	-0.085327	0.073763	
temp	-0.064923	-0.077562	-0.109574	0.052374	
feelslikemax	-0.085529	-0.097952	-0.128615	0.038459	
feelslikemin	-0.078937	-0.072056	-0.091918	0.077461	
feelslike	-0.082353	-0.089657	-0.117679	0.056272	
dew	-0.029542	-0.042229	-0.080179	0.093673	
humidity	0.116130	0.060878	0.071579	0.120122	
precip	-0.036124	-0.018909	0.064665	-0.005481	
precipcover	0.095098	-0.024737	0.151140	-0.018470	
snow	0.325515	0.180149	0.902215	0.015931	
snowdepth	0.075807	0.318364	0.194361	-0.103102	
cloudcover	0.173615	0.049647	0.098584	-0.044717	
uvindex	-0.102558	0.001771	-0.043864	-0.068000	
severerisk	0.015216	0.036830	0.068074	-0.002036	
num_date	-0.052981	-0.075002	-0.106552	0.956032	
preciptype_freezingrain	0.004146	-0.002324	-0.014497	0.042705	
preciptype_freezingrain,snow	-0.039501	-0.028850	-0.001230	-0.055122	
preciptype_freezingrain,snow,ice	-0.026045	-0.009604	0.227668	-0.010105	
preciptype_none	-0.246100	-0.158905	-0.024439	0.057197	

preciptype_rain	-0.260707	-0.132123	-0.070301	0.002156
preciptype_rain,freezingrain	-0.005190	-0.001270	0.089893	-0.054480
preciptype_rain,freezingrain,snow	-0.004840	-0.007330	-0.002236	-0.017704
preciptype_rain,snow	0.924001	-0.080972	0.027369	-0.055376
preciptype_rain,snow,ice	-0.023163	-0.014890	0.132455	-0.022887
preciptype_snow	-0.088322	0.947936	-0.018706	-0.041028

	Factor 9	Factor 10
tempmax	-0.000982	-0.006069
tempmin	0.073334	0.161551
temp	0.031055	0.076709
feelslikemax	-0.005532	-0.014875
feelslikemin	0.056448	0.153186
feelslike	0.020009	0.069495
dew	-0.020443	0.113605
humidity	-0.119973	0.123788
precip	-0.087038	0.001857
precipcover	0.030797	0.100039
snow	-0.120346	-0.009037
snowdepth	-0.063427	0.120553
cloudcover	0.138702	0.143367
uvindex	0.189339	0.039411
severerisk	0.391832	-0.047510
num_date	-0.065573	0.063774
preciptype_freezingrain	0.011394	-0.136688
preciptype_freezingrain,snow	0.067436	-0.115193
preciptype_freezingrain,snow,ice	0.030251	0.003064
preciptype_none	-0.181725	0.151897
preciptype_rain	0.070018	0.138037
preciptype_rain,freezingrain	-0.003426	-0.215202
preciptype_rain,freezingrain,snow	0.053769	-0.056451
preciptype_rain,snow	0.005975	0.054016
preciptype_rain,snow,ice	0.023098	-0.046458
preciptype_snow	0.051313	0.028261

Most Influential Variables in Each Factor:

tempmax	Index(['Factor 1'], dtype='object')
tempmin	Index(['Factor 1'], dtype='object')
temp	Index(['Factor 1'], dtype='object')
feelslikemax	Index(['Factor 1'], dtype='object')
feelslikemin	Index(['Factor 1'], dtype='object')
feelslike	Index(['Factor 1'], dtype='object')
dew	Index(['Factor 1'], dtype='object')
humidity	Index(['Factor 4'], dtype='object')
precip	Index(['Factor 2'], dtype='object')
precipcover	Index(['Factor 2'], dtype='object')
snow	Index(['Factor 7'], dtype='object')
snowdepth	Index(['Factor 1'], dtype='object')

```

cloudcover          Index(['Factor 4'], dtype='object')
uvindex             Index(['Factor 4'], dtype='object')
severerisk          Index(['Factor 9'], dtype='object')
num_date            Index(['Factor 8'], dtype='object')
preciptype_freezingrain Index(['Factor 10'], dtype='object')
preciptype_freezingrain,snow Index(['Factor 10'], dtype='object')
preciptype_freezingrain,snow,ice Index(['Factor 7'], dtype='object')
preciptype_none      Index(['Factor 3'], dtype='object')
preciptype_rain       Index(['Factor 3'], dtype='object')
preciptype_rain,freezingrain Index(['Factor 10'], dtype='object')
preciptype_rain,freezingrain,snow Index(['Factor 10'], dtype='object')
preciptype_rain,snow  Index(['Factor 5'], dtype='object')
preciptype_rain,snow,ice Index(['Factor 7'], dtype='object')
preciptype_snow       Index(['Factor 6'], dtype='object')
dtype: object

```

```

/Users/hunterberberich/anaconda3/lib/python3.11/site-
packages/sklearn/metrics/_classification.py:1469: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.

```

```

_warn_prf(average, modifier, msg_start, len(result))

```

```

/Users/hunterberberich/anaconda3/lib/python3.11/site-
packages/sklearn/metrics/_classification.py:1469: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.

```

```

_warn_prf(average, modifier, msg_start, len(result))

```

```

/Users/hunterberberich/anaconda3/lib/python3.11/site-
packages/sklearn/metrics/_classification.py:1469: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.

```

```

_warn_prf(average, modifier, msg_start, len(result))

```

Logistic Regression Model

Accuracy: 0.8355640535372849

Classification Report:

	precision	recall	f1-score	support
0.0	0.00	0.00	0.00	172
1.0	0.84	1.00	0.91	874
accuracy			0.84	1046
macro avg	0.42	0.50	0.46	1046
weighted avg	0.70	0.84	0.76	1046

Confusion Matrix:

```

[[ 0 172]
 [ 0 874]]

```

Random Forest Model

Accuracy: 0.7896749521988528

Classification Report:

	precision	recall	f1-score	support
0.0	0.15	0.06	0.08	172
1.0	0.83	0.93	0.88	874
accuracy			0.79	1046
macro avg	0.49	0.50	0.48	1046
weighted avg	0.72	0.79	0.75	1046

Confusion Matrix:

```
[[ 10 162]
 [ 58 816]]
```

Gradient Boosting Model

Accuracy: 0.8307839388145315

Classification Report:

	precision	recall	f1-score	support
0.0	0.14	0.01	0.01	172
1.0	0.84	0.99	0.91	874
accuracy			0.83	1046
macro avg	0.49	0.50	0.46	1046
weighted avg	0.72	0.83	0.76	1046

Confusion Matrix:

```
[[ 1 171]
 [ 6 868]]
```

SVM Model

Accuracy: 0.8355640535372849

Classification Report:

	precision	recall	f1-score	support
0.0	0.00	0.00	0.00	172
1.0	0.84	1.00	0.91	874
accuracy			0.84	1046
macro avg	0.42	0.50	0.46	1046
weighted avg	0.70	0.84	0.76	1046

Confusion Matrix:

```
[[ 0 172]
```

```
[ 0 874]]
```

```
/Users/hunterberberich/anaconda3/lib/python3.11/site-  
packages/sklearn/metrics/_classification.py:1469: UndefinedMetricWarning:  
Precision and F-score are ill-defined and being set to 0.0 in labels with no  
predicted samples. Use `zero_division` parameter to control this behavior.  
    _warn_prf(average, modifier, msg_start, len(result))  
/Users/hunterberberich/anaconda3/lib/python3.11/site-  
packages/sklearn/metrics/_classification.py:1469: UndefinedMetricWarning:  
Precision and F-score are ill-defined and being set to 0.0 in labels with no  
predicted samples. Use `zero_division` parameter to control this behavior.  
    _warn_prf(average, modifier, msg_start, len(result))  
/Users/hunterberberich/anaconda3/lib/python3.11/site-  
packages/sklearn/metrics/_classification.py:1469: UndefinedMetricWarning:  
Precision and F-score are ill-defined and being set to 0.0 in labels with no  
predicted samples. Use `zero_division` parameter to control this behavior.  
    _warn_prf(average, modifier, msg_start, len(result))
```

```
[ ]: features
```

```
[ ]: Index(['tempmax', 'tempmin', 'temp', 'feelslikemax', 'feelslikemin',  
          'feelslike', 'dew', 'humidity', 'precip', 'precipcover', 'snow',  
          'snowdepth', 'cloudcover', 'uvindex', 'severerisk', 'num_date',  
          'preciptype_freezingrain', 'preciptype_freezingrain,snow',  
          'preciptype_freezingrain,snow,ice', 'preciptype_none',  
          'preciptype_rain', 'preciptype_rain,freezingrain',  
          'preciptype_rain,freezingrain,snow', 'preciptype_rain,snow',  
          'preciptype_rain,snow,ice', 'preciptype_snow'],  
          dtype='object')
```

```
[ ]:
```

```
[ ]:
```